

遥感目标检测—高级基础知识与流程（PDF 版）

May 21, 2026

说明

本文档以中文详细说明：开放提示词、数据集规模与 GPU 估计、上/下采样、IoU/GIoU 用法、NMS、伪标签置信度与一致性方法、batch 含义与影响，以及从输入到输出的完整流程并给出数值示例。公式均使用 LaTeX 书写以确保可读与可编译。

1 什么是开放提示词（Open Prompt / Prompt）

开放提示词（Prompt）最初来自语言模型，是一段用于引导模型产生特定行为的“提示”。在遥感检测中，开放提示词可以扩展为：类别描述、场景元信息、可学习的条件向量或生成器的控制向量。

数学上，如果模型为 $f(x; \theta, p)$ ，其中 x 为图像， θ 为模型权重， p 为 prompt，则不同 prompt 会得到不同条件预测：

$$y_p = f(x; \theta, p).$$

当我们拥有一个 prompt 集合 $P = \{p_1, \dots, p_N\}$ （即 prompt-bank）时，可按不同策略使用它：平均化、加权或检索最优 prompt。详见后文“Prompt-bank”部分。

2 数据集大小与预计 GPU 耗费

数据集举例

- DOTA v2（近似规模）：约一万级原始图像；通过瓦片化（例如每图切 1024x1024 瓦片）后，瓦片总数常扩增到数万至十万级（取决于 tile stride 与 overlap）。

GPU 内存估计（经验值）

给定配置：Backbone=ResNet-50，输入 tile=1024×1024，FPN，单阶段 Head。显存估计（近似）：

- batch size per GPU = 1：需要约 8–10 GB；
- batch size per GPU = 2：需要约 12–16 GB；
- batch size per GPU = 4：需要约 20–28 GB。

估计原因：特征图分辨率大，FPN 与 detection head 会在多尺度上保存中间激活，导致显存占用迅速上升。若使用更轻的 backbone（MobileNet、EfficientNet-lite）或把 tile 缩小到 512，会显著降低显存需求。

如果使用多 GPU（例如 4 卡），可以把全局 batch size 分摊到每卡，从而在相同学习率策略下提高吞吐量。

3 上采样与下采样对应什么

在卷积神经网络中：

- 下采样 (downsampling) 通常意味着空间分辨率下降, 常见操作: 步幅为 2 的卷积 (stride=2)、池化 (max/avg pool)。下采样增大感受野、降低计算与内存, 但会损失高频/细节信息。
- 上采样 (upsampling) 为把低分辨率特征恢复到更高分辨率, 常见操作: 最近邻/双线性插值、反卷积 (transposed convolution)、子像素重排 (pixel shuffle)。FPN 中常用上采样把深层语义向浅层传递。

数学上, 若 $F \in \mathbb{R}^{H \times W \times C}$ 为低分辨率特征, 上采样 $U(\cdot)$ 会输出 $\tilde{F} = U(F) \in \mathbb{R}^{\alpha H \times \alpha W \times C'}$, 其中 α 是放大倍数 (通常 $\alpha = 2$)。

4 IoU 和 GIoU 在哪里用到

IoU (Intersection over Union)

IoU 用于:

- 训练阶段衡量 anchor 与 GT 的匹配 (Assignment);
- 评估指标 (检测 IoU 阈值如 AP@0.5);
- 后处理 (NMS 中计算两个预测框是否重复)。

IoU 定义为:

$$\text{IoU}(A, B) = \frac{\text{area}(A \cap B)}{\text{area}(A \cup B)}.$$

GIoU (Generalized IoU)

GIoU 作为 IoU 的改进, 常用于目标回归损失以获得更稳定的梯度, 定义:

$$\text{GIoU}(A, B) = \text{IoU}(A, B) - \frac{\text{area}(C) - \text{area}(A \cup B)}{\text{area}(C)},$$

其中 C 是包含 A 和 B 的最小闭包框。GIoU 常被用作回归损失的替代 (或混合) 以提升在非重叠框情形下的收敛性。

在代码中: IoU 用于 assignment 与 NMS, GIoU (或 CIoU/DIoU) 用于回归损失项 L_{box} 。

5 NMS (Non-Maximum Suppression) 详解

NMS 的目的: 在推理阶段去除对同一目标的重复检测。算法流程:

1. 对预测框按置信度 s_i 降序排序;
2. 取最高分框 b_1 放入保留集合;
3. 从剩余盒子中删除与 b_1 的 $\text{IoU} > T_{nms}$ 的盒子;
4. 重复步骤 2-3 直到没有框。

直观上, NMS 认为若两个框高度重合 (IoU 很大), 它们很可能是对同一目标的重复检测, 只保留得分更高者。阈值 T_{nms} 通常取 0.4-0.6。

Soft-NMS 替代硬删除为按 IoU 衰减分数: 若框 b_j 与当前保留框 b_k 的 IoU 为 o , 则把 s_j 更新为 $s_j \leftarrow s_j \cdot f(o)$, 常用的衰减函数是高斯或线性函数; 这样在密集目标场景下更稳健。

举例: 两个框置信度分别 0.95 和 0.9, IoU=0.7, 若 $T_{nms} = 0.5$, 则第二个框被删除; 在 Soft-NMS 中第二个框的分数会衰减到例如 0.5, 但仍可能被保留 (取决于阈值)。

6 伪标签置信度、阈值、模型一致性与时序教师（Mean Teacher）

伪标签置信度如何计算

对每个预测框，模型通常输出类别概率向量 $p = (p_1, \dots, p_C)$ ，置信度可取为：

$$s = \max_k p_k.$$

或取置信度为正类概率 $p_{\hat{y}}$ ，其中 $\hat{y} = \arg \max_k p_k$ 。对于多任务（回归 + 分类），也可用 $s = p_{\hat{y}} \cdot f_{box}(\text{box quality})$ 。

阈值设定

阈值 τ 一般为经验设定（常见 0.5–0.95），更严格的阈值（如 0.9）能降低伪标签噪声但减少被采纳的样本数量。选择策略：

- 固定阈值：简单直接；
- 动态阈值：按每类置信度分布自适应（例如每类取 top-K 或按分位数设阈）；
- 质量加权：同时考虑类别置信度与 bbox 质量（IoU/置信度乘积）。

若 $s \geq \tau$ 则该预测可作为伪标签（ $\tilde{y}$ ）被加入训练集中，但通常还会为伪标签评分加权，即训练损失乘以权重 $w(s)$ （例如 $w(s) = s$ 或常数小权重）。

模型一致性

模型一致性指多个预测来源之间的一致程度：

- 多模型一致性：用多个不同初始化/架构/检查点的模型预测同一无标签样本，只有当多数模型给出相似预测（位置与类别）时才接纳伪标签；
- 数据增强一致性：对同一图像做不同增强，若预测一致则说明可信度更高。

时序教师（Mean Teacher）

Mean Teacher 方法维持一对模型：student 与 teacher。teacher 的权重 $\theta^{(t)}$ 通过 student 的历史权重指数移动平均（EMA）更新：

$$\theta_{teacher}^{(t)} \leftarrow \alpha \theta_{teacher}^{(t-1)} + (1 - \alpha) \theta_{student}^{(t)},$$

其中 $\alpha \in [0, 1)$ （例如 0.99）。teacher 通常比 student 更稳定，作为伪标签生成器提供更可靠的目标预测。

7 Batch 的含义与对硬件与性能的影响

定义

Batch（批次）是指一次前向/反向传播中同时处理的样本数量。若全局 batch size 为 B ，每个 GPU 上的 batch size 为 $B_{gpu} = B / \#GPU$ （在数据并行时）。

影响

- 显存需求：与 batch 大小成正比；
- 梯度估计方差：较大 batch 可降低梯度噪声，训练更稳定；
- 学习率关系：线性缩放规则（Linear Scaling Rule）建议当 batch 成倍增加时相应放大学习率： $\eta' = \eta \cdot (B'/B)$ 。但过大 batch 会导致泛化性下降（需调参）；
- 吞吐量与速度：更大 batch 可提高 GPU 并行利用率，但受限于显存与 I/O。

经验建议：在 1–8 的每卡 batch 范围内，通常可获得稳定训练与可控显存；对大尺度检测任务，每卡 batch 经常为 1 或 2。

8 完整流程（从输入到输出）与具体数值示例

下面按紧密逻辑顺序列出完整流程，并给出一个数值化的例子（便于理解）。

流程步骤

1. 数据准备：原始大图（例如 2000 张，每张 6000×6000 ）按 $\text{tile}=1024$ 、 $\text{stride}=768$ 切片，得到约 $\sim 15\text{k}$ 瓦片，标注转换为瓦片坐标；
2. 数据增强：随机水平/垂直翻转、随机尺度 (0.8–1.2)、颜色扰动；
3. Backbone 提取特征：ResNet-50 输出层级特征 C_3, C_4, C_5 ；
4. Neck (FPN)：融合得到多尺度特征 P_3, P_4, P_5 ；
5. Head (Anchor-based)：在每个位置放置 anchors（例如 $\text{scales} \{32, 64, 128\}$ 和 $\text{ratios} \{0.5, 1, 2\}$ ），预测分类概率与回归偏移；
6. Assignment：计算 anchor 与 GT 的 IoU，正负样本按 $T_{pos} = 0.5, T_{neg} = 0.4$ 分配；
7. 损失计算与优化：分类用 Focal Loss ($\gamma = 2, \alpha = 0.25$)，回归用 Smooth L1 或 GIoU，优化器用 SGD ($\eta = 0.01, \text{momentum}=0.9$)；
8. 推理后处理：对所有预测应用阈值筛选（例如 $s \geq 0.05$ ），再执行 NMS ($T_{nms} = 0.5$)；
9. 评估：计算 AP@50、AP@75 及 mAP；
10. 若使用自训练：用当前 checkpoint 预测无标签数据，筛选 $s \geq \tau$ （例如 $\tau = 0.9$ ）的伪标签并加入训练或加权训练；重复训练。

数值例子（端到端）

设初始条件：2000 原图，tile 得到 15000 瓦片；使用 2 张 16GB GPU，ResNet-50， $\text{tile}=1024$ ， $\text{batch per GPU}=2$ （全局 $\text{batch}=4$ ），训练 12 个 epoch。

训练期间的典型超参：

$$\eta = 0.01, \text{momentum} = 0.9, \text{weight_decay} = 1e - 4.$$

每次迭代计算：

- 正样本数 N_{pos} 在 batch 内波动，例如平均为 32；
- 分类损失为 Focal Loss，总体值随训练下降（示例：初始 1.8 -> 收敛 0.12）；
- 伪标签阶段：用 teacher 生成预测并以 $\tau = 0.9$ 筛选，假设在 5000 张无标签瓦片上得到 12000 个高置信伪标签，按权重 $w(s) = s$ 加入训练。

最终输出：模型 checkpoint（权重文件），以及评估结果表（AP 各类与 mAP）。